Audio Recorder - Part 2

05

It was so easy to create the functionality of the audio recorder app with coding. Now, I must ensure that the app functionality can be understood by the audience!

For audience to easily understand how to use the app, it needs to be simple and user-friendly. This will ensure that users select this app over other more complicated ones available in the market.

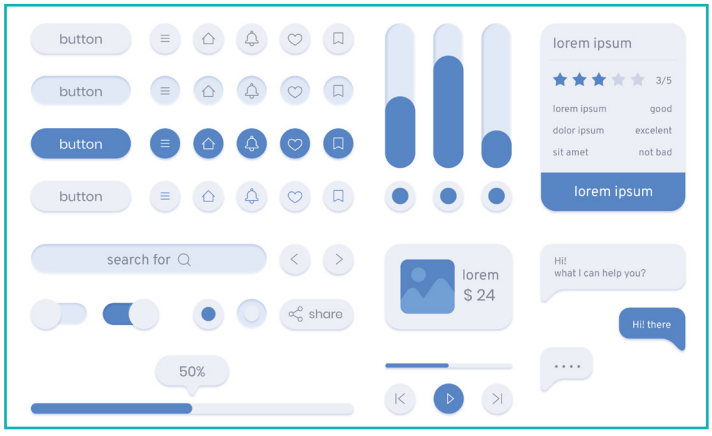
In the previous session, we learnt the basic functionalities of audio recording. In this session, we will ensure that the app is user-friendly. For that, the display and functions need to be easy to understand and follow. This is where the User Interface or UI comes in.

5. Audio Recorder - Part 2

User Interface (UI)

'User interface' refers to the graphical layout of the software or computerised devices that are designed with an emphasis on looks, style, readability, and usability for the user. This is the window through which the user interacts with an application.

A User Interface consists of items used to display elements of an app. They can consist of buttons, text, images, sliders, various types of input fields, etc. The UI also focusses on the strategic placement of these elements in a way that is attractive to the user. The process of designing a user interface is called '**User Interface Design'.**



User Experience (UX)

The 'User Experience' of an app defines how a user feels while interacting with elements of an app. The ease of access is determined by how well the interface is designed for an effective user experience.

5. Audio Recorder - Part 2

A successful UX design comprises intuitive user interaction, easy navigation through the elements of the app, and the ease of operating the app with minimal instructions.

UX and UI design of an app are co-dependent. While the UI refers to the visual aspects of the app that the user interacts with, UX refers to the overall ease of access to the app. Both elements together create a satisfactory user experience.

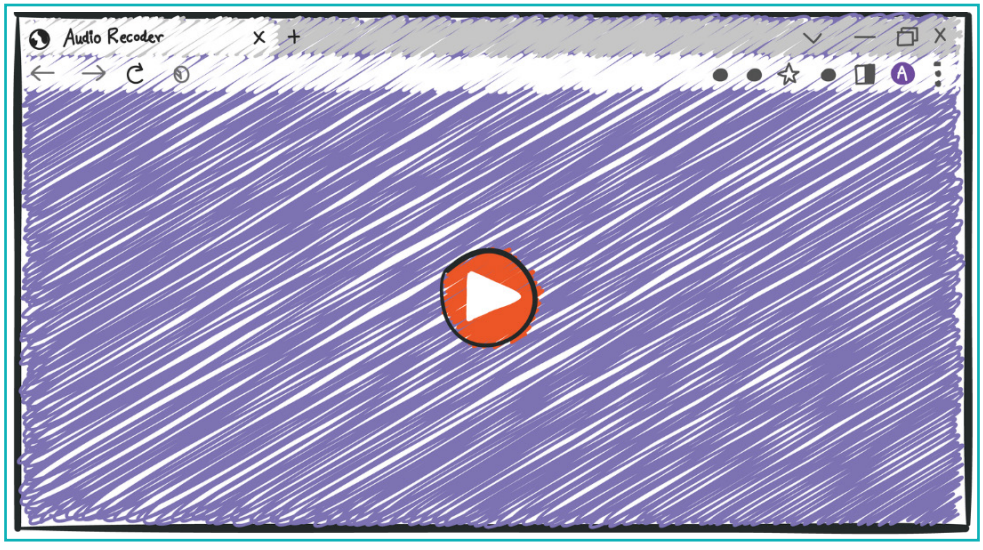
Now that we are clear about the user element of the app, we will create a simple but aesthetic UI and an intuitive UX for the audio recorder app we created in the previous lesson.

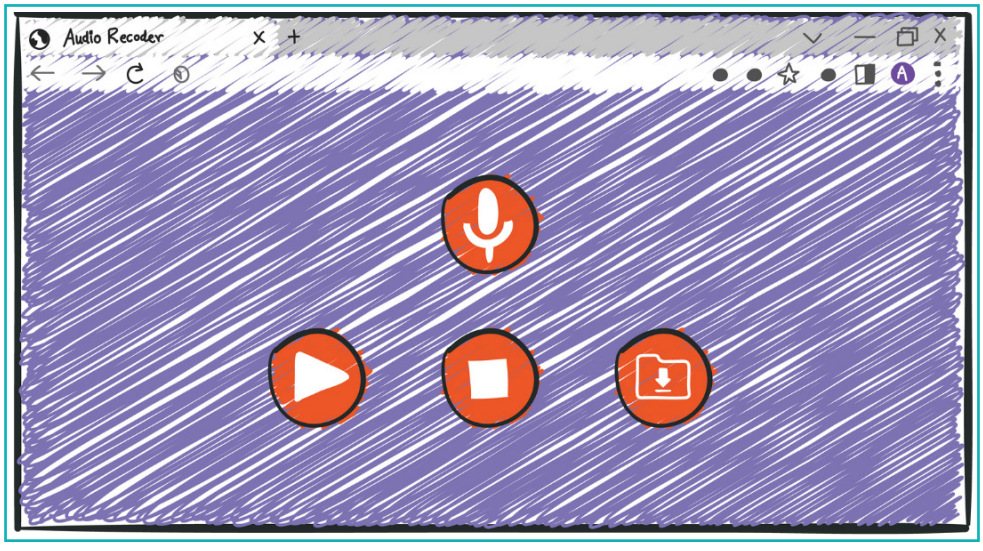
The elements we need in an audio recorder app are:

1. Interface background

2. Buttons for play, stop, and save the file.

We can visualize the interface as follows:





Exercise

Fill in the blanks.

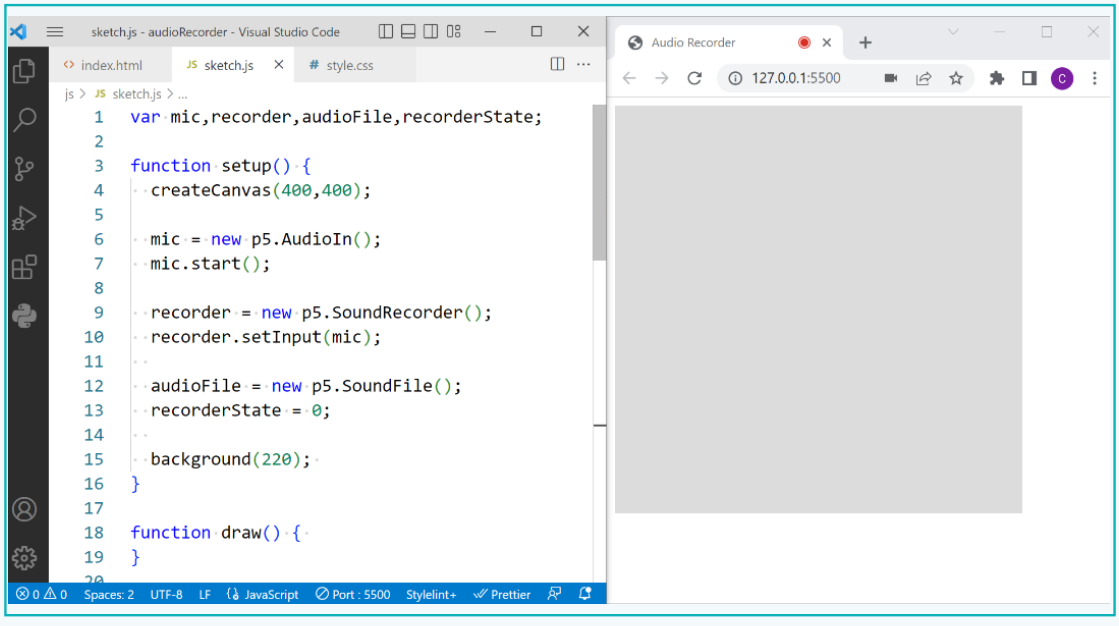
1. _____ is the graphical layout of an app or a device designed considering looks, style, readability, and usability of the user.
2. _____ is how the user feels while interacting with various elements of the app.

Activity

To simplify the working of the app, we will only display one button at once. For example, when we start recording, it is unnecessary to display the 'record' button. It will be replaced by the 'stop' button which will be used to stop recording.

The app we created to record audio only had functionality and no user interface. Let's update it.

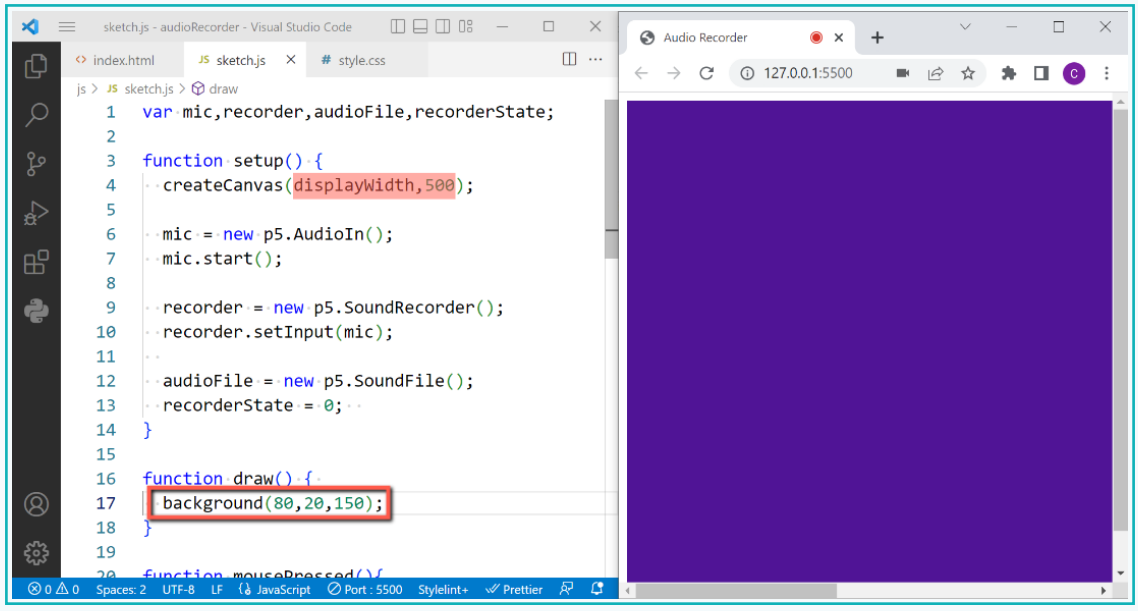
- Open the activity we created in the last lesson.



Activity

Step 1: Set the app interface, background size, and colour

1. Set the width of the canvas as ‘displayWidth’ for the full width and height as ‘500px’.
2. Set the background colour of the canvas: ‘R=80’, ‘G=20’, ‘B=150’.



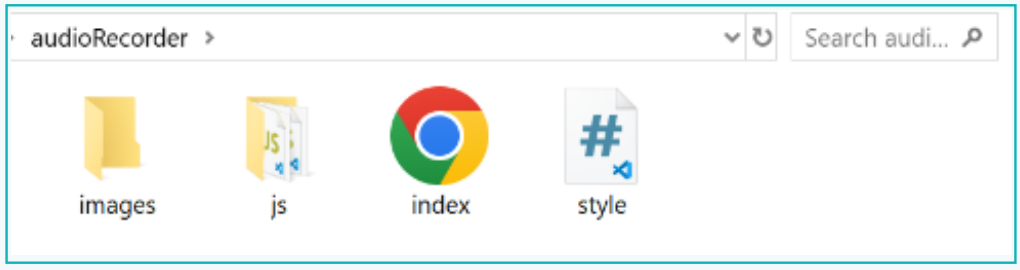
Step 2: Find Buttons Icon

To operate the app, we need to add buttons. **‘Buttons’** are represented using icon images. We can find icons for ‘Record’, ‘Stop’, and ‘Save’ online for free download. We can upload images as ‘Html image’ elements and use them as a button.

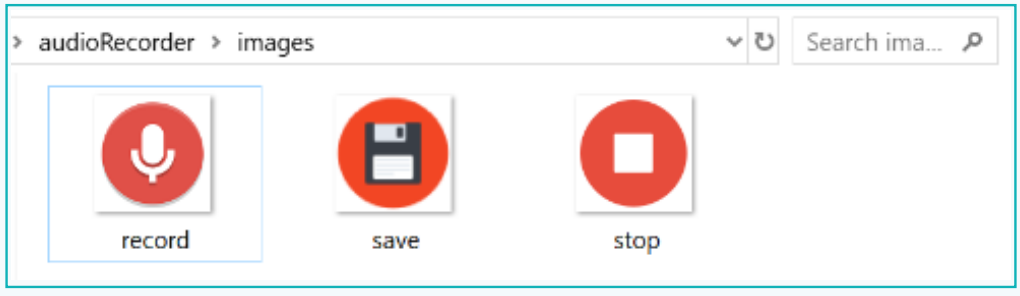
1. Find icons for recording, stop, and save buttons online and download them. You can also use this QR code to use the icons used in this activity.

Activity

2 Create a folder named “images” in the project folder.



2 Add the downloaded button icon images in the “images” folder.



To use the image as a button, we need to load and display the image in the code using a variable and the ‘createImg()’ command.

Step 3: Add button image

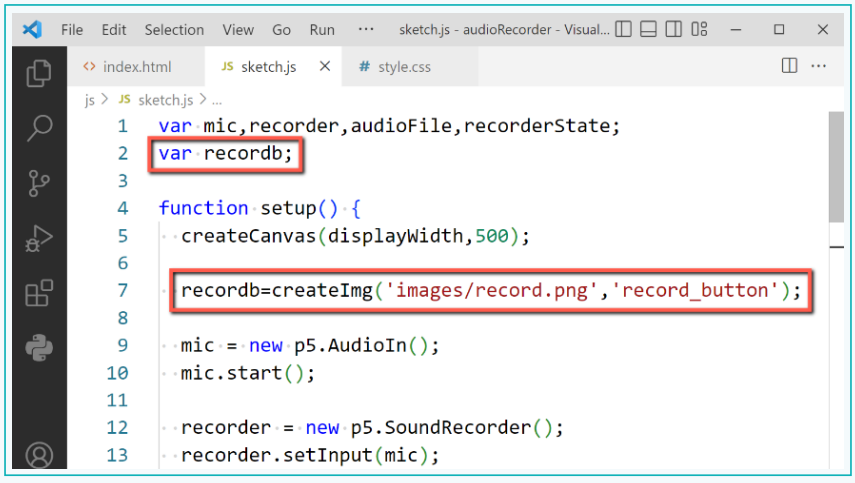
The ‘createImg(src, alt)’ command creates an <img> html element in the DOM. It uses the given src and alternate text, where the src path or URL is used for the image, and the alternate text is the text to be displayed if the image does not load.

1 Create a variable at the top named ”recordb” to store the record button image.

2 Use the ‘createImg(‘path’, alt)’ command to upload the image and assign it to the ‘recordb’ variable.

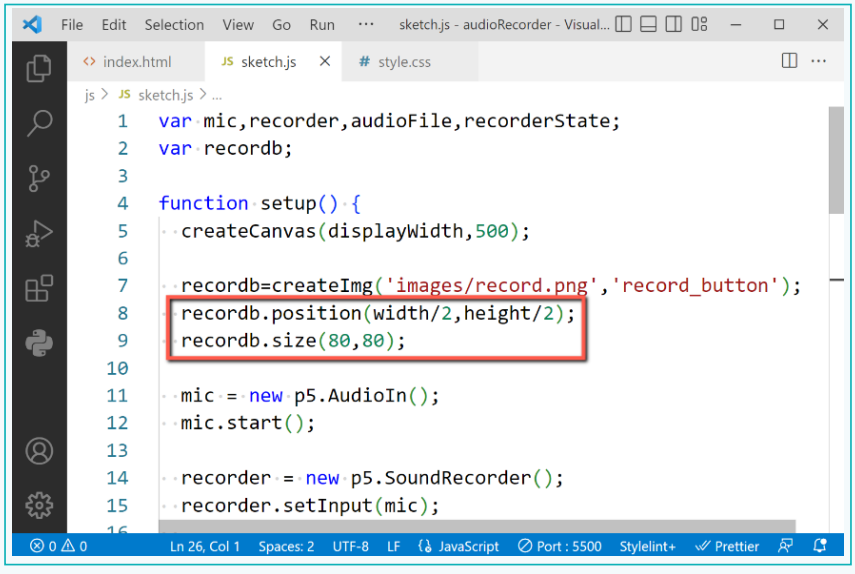
Activity

3. Add the first parameter as the path to the image and the second parameter as the alternative text for the image in case it is not loaded.



4. Use the 'position()' method with dot operator for record button 'recordb' to set the position of the image at the center of the canvas.

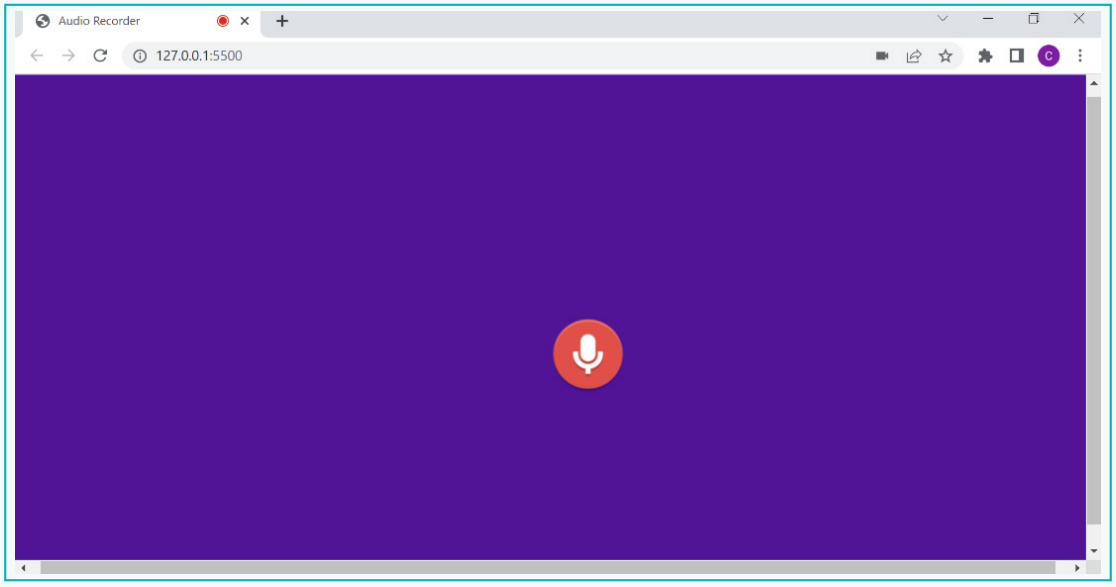
5. Use the 'size()' method to set the size of the button image to '80 x 80 pixels'.



6. Run the code by activating the local live server.

Activity

We will be able to see the record button image at the centre of the browser window. Note that the images used as buttons must have a transparent background to ensure that only the button is visible on the screen.



7. Add the 'stop' button using the 'stopb' variable and 'createImg' command, just like the record button of the same size at the same position.

8. Use the 'hide()' method to initially hide the stop button.

```
3
4   function setup() {
5     createCanvas(displayWidth,500);
6
7     recordb=createImg('images/record.png','record_button');
8     recordb.position(width/2,height/2);
9     recordb.size(80,80);
10
11    stopb=createImg('images/stop.png','stop_button');
12    stopb.position(width/2,height/2);
13    stopb.size(80,80);
14    stopb.hide();
15
16    mic = new p5.AudioIn();
```

Activity

9 Similarly, create the 'save' button and hide it initially.

```
function setup() {
  createCanvas(displayWidth,500);

  recordb=createImg('images/record.png','record_button');
  recordb.position(width/2,height/2);
  recordb.size(80,80);

  stopb=createImg('images/stop.png','stop_button');
  stopb.position(width/2,height/2);
  stopb.size(80,80);
  stopb.hide();

  saveb=createImg('images/save.png','save_button');
  saveb.position(width/2,height/2);
  saveb.size(80,80);
  saveb.hide();
```

As we will use the images as buttons to operate the recording app, remove the code added earlier using the 'mousePressed()' button. Instead of deleting it, just comment it out, as we can use some snippeets of the code inside.

10 Comment out the code for the entire 'mousePressed()' function.

sketch.js - audioRecorder - Visual Studio Code

index.html | sketch.js | style.css

js > sketch.js > ...

```
// function mousePressed(){
//   userStartAudio();
//   //console.log('click');
//   if(recorderState == 0){
//     if(mic.enabled){
//       recorder.record(audioFile);
//       recorderState = 1;
//       background(255,0,0);
//     }
//   }else if(recorderState == 1){
//     recorder.stop();
//     recorderState = 2;
//     background(0,255,0);
//   }else if(recorderState == 2){
//     saveSound(audioFile,'mySound.wav');
//     recorderState = 0;
//     background(220);
//   }
```

Activity

As we want to use an image element as a button, we can use the 'mouseClicked' method.

The **'mouseClicked(fxn)'** method is called once a mouse button is pressed and then released over the element like an image, where 'fxn' is the function that will be executed when the mouse clicks on the element.

The function inside the braces is invoked as a result of the mouse clicking on the element.

Step 4: Invoke Function

1. Use the 'mouseClicked()' method for the record button and invoke the 'startRecord' function, in the draw().
2. Create a new function 'startRecord' after the draw.

```
31  function draw() {
32    background(80,20,150);
33
34    recordb.mouseClicked(startRecord);
35  }
36
37  function startRecord(){
38
39  }
40
```

3. Add the 'userStartAudio()' command as added earlier in the 'mousePressed()' function.
4. Add the code lines inside the condition of **(recorderState == 1)** in the commented code, as the functionality of the 'record' button.
5. We can skip the code line to change the background colour to 'red'.

```
36
37  function startRecord(){
38    userStartAudio();
39    if(mic.enabled){
40      recorder.record(audioFile);
41      recorderState = 1;
42    }
43  }
```

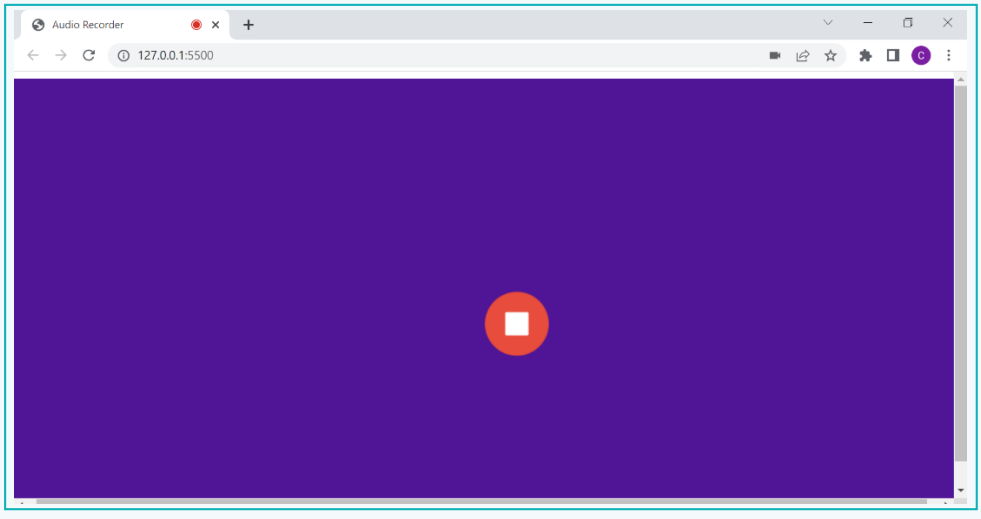
Activity

6. Add the ‘recordb.hide()’ command to hide the record button once the recording starts.

7. Add ‘stopb.show()’ to display the stop button to able to stop recording.

```
37  function startRecord(){
38      userStartAudio();
39      if(mic.enabled){
40          recorder.record(audioFile);
41          recorderState = 1;
42          recordb.hide();
43          stopb.show();
44      }
```

8. Check the code by pressing the ‘record’ button. We can see that the ‘record’ button will disappear, and the ‘stop’ button will appear. Also, the red dot on the tab starts blinking as an indication of recording.



9. Just like the record button, use the ‘mouseClicked’ method for the ‘stopb’ variable in the draw() to invoke the ‘stopRecord’ function.

10. Create a ‘stopRecord’ function and add code to stop the recording using the ‘recorder.stop()’ method.

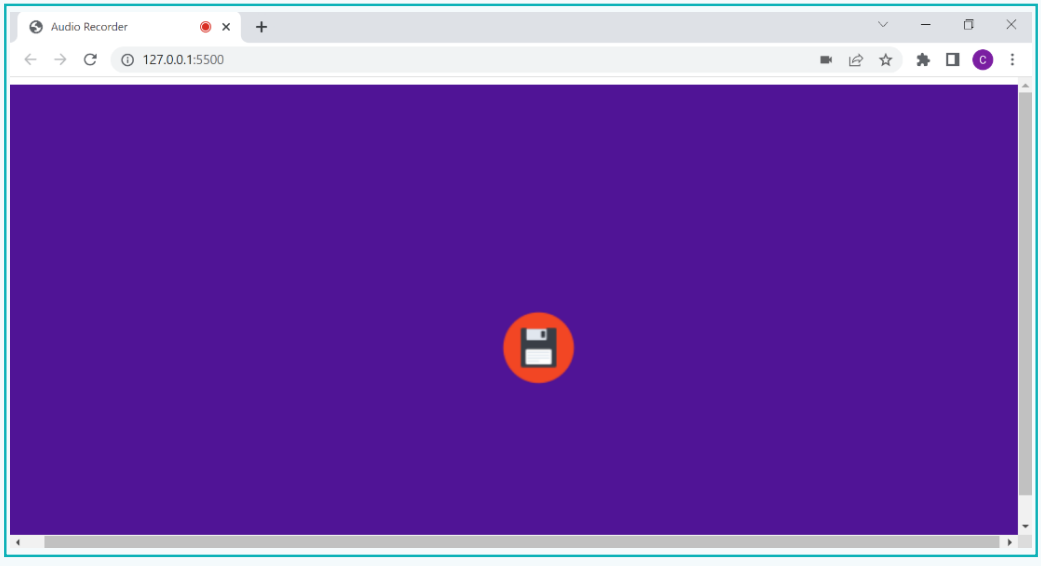
11. Set the ‘recorderState’ variable value to ‘0’ as the recording is stopped.

Activity

12 Hide the ‘stop’ button once clicked and display the ‘save’ button to save the recorded audio using the ‘hide’ and ‘show’ methods, respectively.

```
34    recordb.mouseClicked(startRecord);
35    stopb.mouseClicked(stopRecord);
36  }
37
38  function stopRecord(){
39    recorder.stop();
40    recorderState=0;
41    stopb.hide();
42    saveb.show();
43  }
44
```

13 Run the code to check if the recording stops when the ‘stop’ button is clicked, and the ‘save’ button appears.

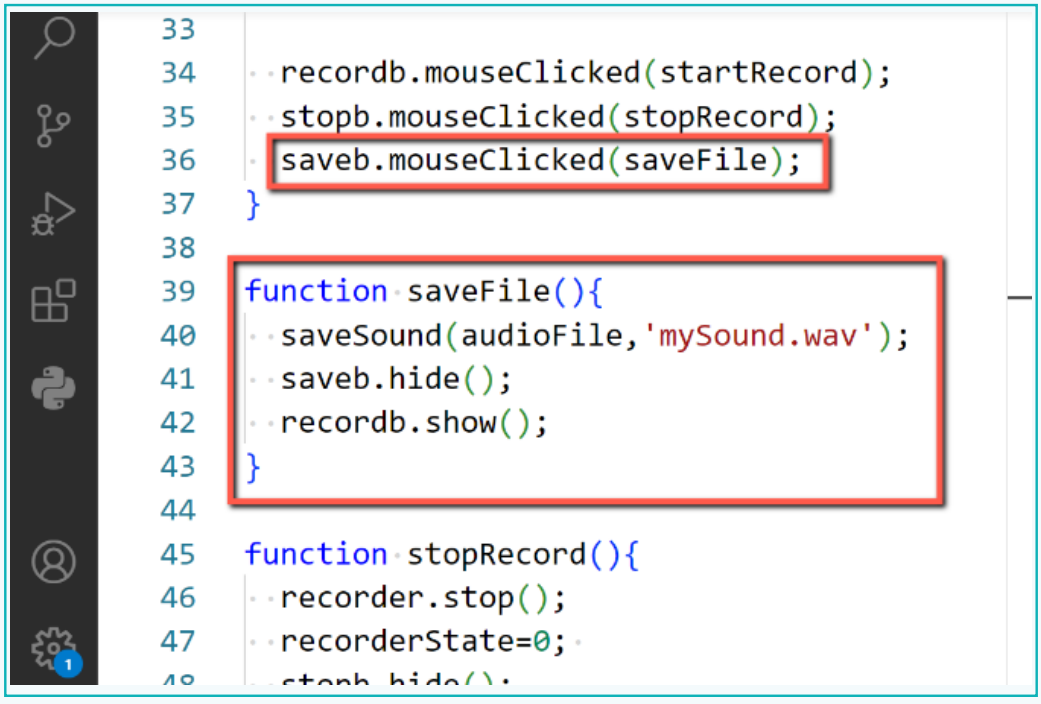


14 Use the ‘mouseClicked’ method for the ‘saveb’ variable in the draw() to invoke the ‘saveFile’ function.

15 Create a ‘saveFile’ function and add code to save the ‘audioFile’ as a ‘mySound.wav’ file.

Activity

16 Hide the 'save' button and display the 'record' button once the audio has been saved. This allows the user to be able to record new audio, if required.



```
33
34    recordb.mouseClicked(startRecord);
35    stopb.mouseClicked(stopRecord);
36    saveb.mouseClicked(saveFile);
37  }
38
39  function saveFile(){
40    saveSound(audioFile,'mySound.wav');
41    saveb.hide();
42    recordb.show();
43  }
44
45  function stopRecord(){
46    recorder.stop();
47    recorderState=0;
```

17 Click on the 'save' button to check the code. A pop-up window will appear, allowing you to select a location to save the recorded audio file.

With this, the basic functionality of the audio recorder is completed. Now we can add some additional app features like title and sound visualization.

Step 5: Add the App Title

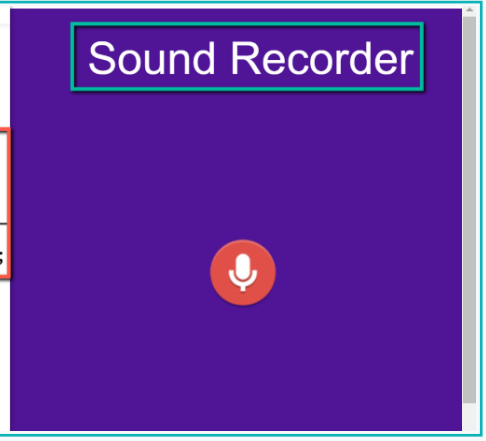
1 Add the text 'Sound Recorder' at 'x = width/2 +40' and 'y = 70' to place it at the top center position of the canvas.

2 Use the 'textAlign' property and set it to 'centre', such that the centre point of the text window is used to position the text.

Activity

3. Add commands to set the 'textSize' to '48', fill the colour as 'white', and have 'no border' to the text.

```
30
31  function draw() {
32    background(80,20,150);
33
34    textAlign(CENTER);
35    textSize(48);
36    fill(255);
37    noStroke();
38    text('Sound Recorder',width/2+40,70);
39
40    recordb.mouseClicked(startRecord);
41    stopb.mouseClicked(stopRecord);
42    saveb.mouseClicked(saveFile);
43  }
44
```



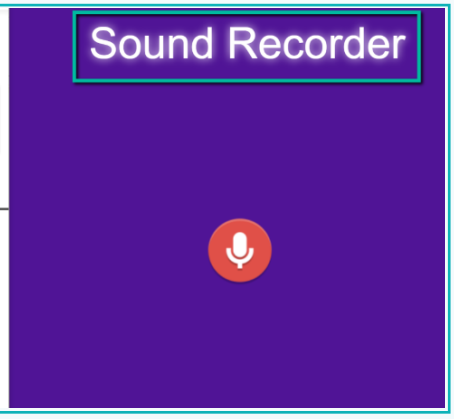
The text will be displayed as an app title. We can also enhance its look as a part of the UI Design for it to be aesthetically pleasing. We can add a glow effect to the text using features from HTML.

The p5.js provides a lot of functionality for creating graphics. Some HTML canvas functionalities, like adding shadow effects, blur effects, etc. are not available, at present, in p5js. We can still use these functionalities in p5js projects using the 'drawingContext' default object in JavaScript, as p5js code is essentially a JavaScipt code. Therefore, all language features and libraries other than the p5 library can be used. For example, we can set the 'shadowBlur' and 'shadowColor' properties of 'drawingContext' to add a shadow effect.

Step 6: Add a Shadow Effect

1. Add the 'drawingContext.shadowBlur' property to '20' before adding the text.
2. Set 'drawingContext.shadowColor' to 'white'.

```
31  function draw() {
32    background(80,20,150);
33
34    drawingContext.shadowBlur = 20;
35    drawingContext.shadowColor = 'white';
36
37    textAlign(CENTER);
38    textSize(48);
39    fill(255);
40    noStroke();
41    text('Sound Recorder',width/2+40,70);
42
43    recordb.mouseClicked(startRecord);
44    stopb.mouseClicked(stopRecord);
45    saveb.mouseClicked(saveFile);
```



We can see that the glow effect is added to the text.

5. Audio Recorder - Part 2

Exercise

State if the statements are True or False

3. The ‘createImg()’ command takes two input parameters, with the first as a path to the image file and the second as alternate text to be displayed.

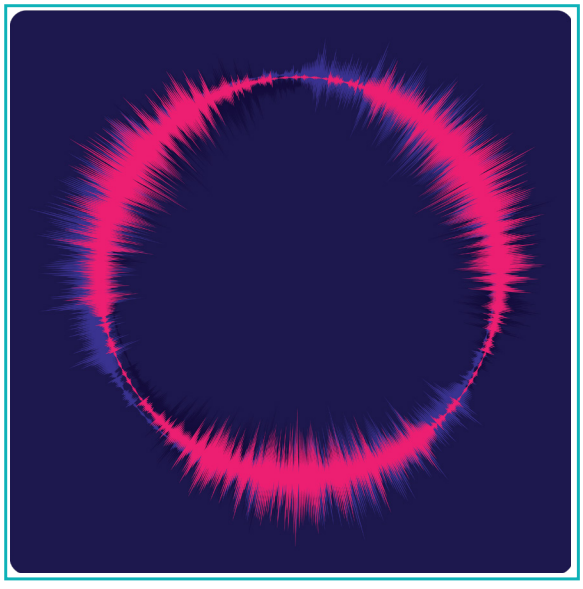
   _____

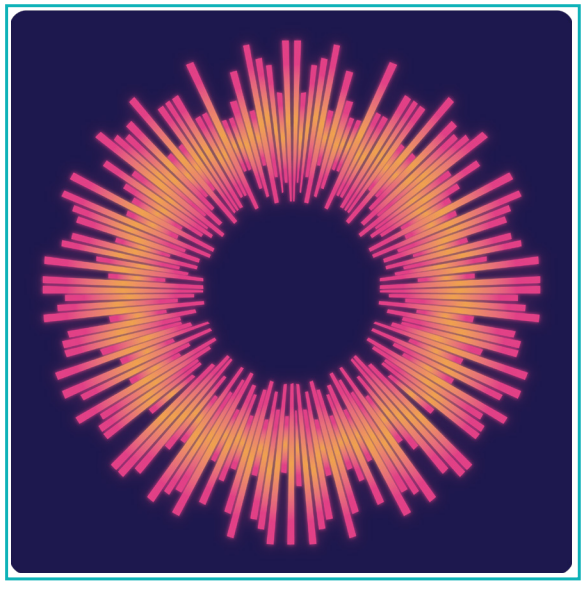
4. The ‘mouseClicked(fxn)’ method is used to check if an element like an image is clicked and if yes, the ‘fxn’ function is invoked.

   _____

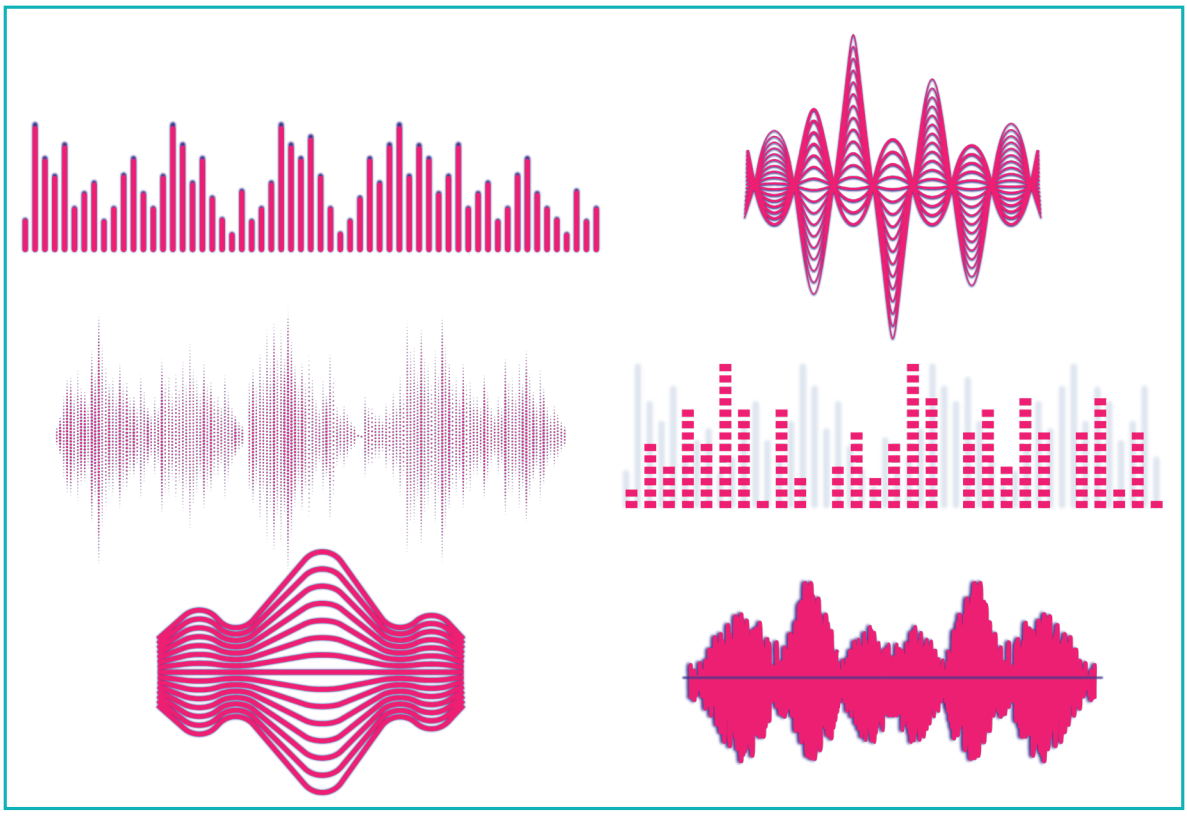
It’s a good feature to highlight when the recording is on. We can show this using sound visualization when the audio is recorded. **Sound** or **Audio Visualization** is the process of converting sound into visually perceptible images or animation.

The amplitude or frequency of the recorded audio is used in many instances to show the visualization of sound.





5. Audio Recorder - Part 2



We will implement a simple sound visualization in which the amplitude of the recorded sound will be mapped on the diameter of the circle. As the audio is being recorded, a circle with continuously varying diameters will be displayed.

Activity

Step 7: Create the Sound Visualization

1. Add the *'if'* condition to check when the 'recorderState' variable value is '1', i.e., recording is on in draw().
2. Use the 'mic.getLevel()' method to get the amplitude of the audio being recorded by the microphone and save it in a local variable 'amp'.

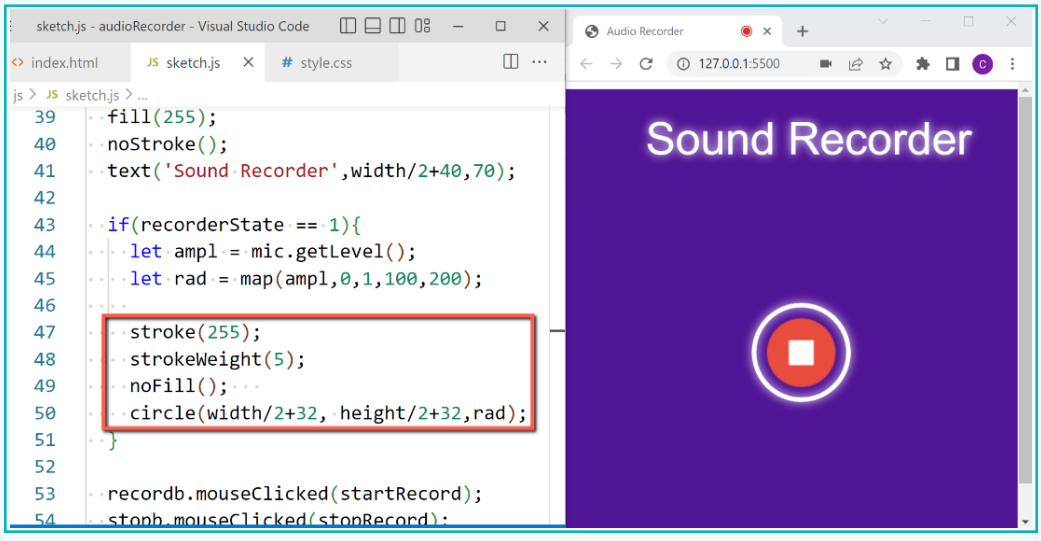
Activity

3. Use the 'map()' command to convert the amplitude value between the range of '0-1' to '100-200' as the 'circle' diameter and save it to the local variable 'rad'.

```
37  textAlign(CENTER);
38  textSize(48);
39  fill(255);
40  noStroke();
41  text('Sound Recorder',width/2+40,70);
42
43  if(recorderState == 1){
44    let ampl = mic.getLevel();
45    let rad = map(ampl,0,1,100,200);
46  }
47
48  recordb.mouseClicked(startRecord);
```

4. Draw a 'circle' with value in variable 'rad' as diameter, positioned around the button, such that both are concentric with 'white' colour stroke and strokeWeight '5'.

5. Add 'noFill()' command to make the circle transparent.

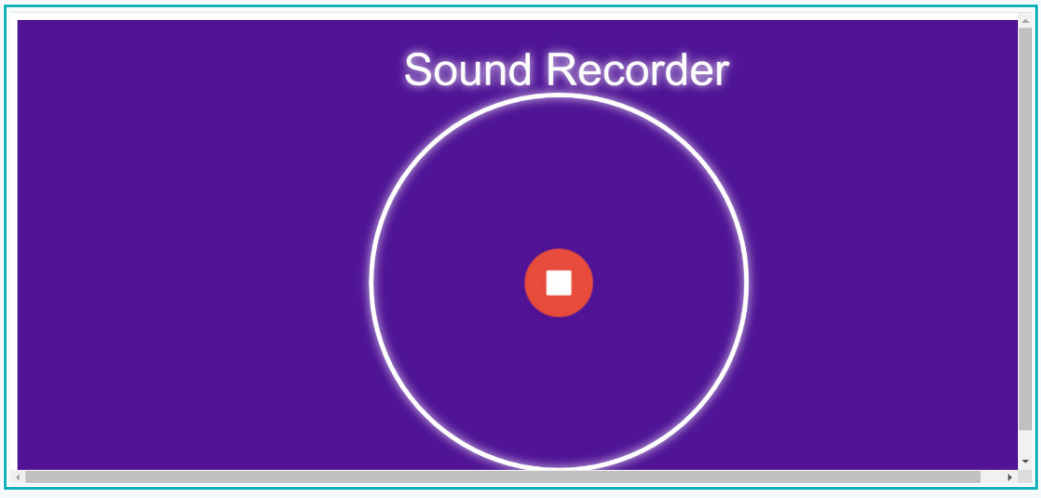


Activity

Note that with this mapping, there will be no significant change in the circle diameter when we start recording. This is because the mapping ranges are not appropriate to enhance the changes in amplitude. We can adjust the value such that the amplitude range of '0-0.2' corresponds to the range of '100-500'.

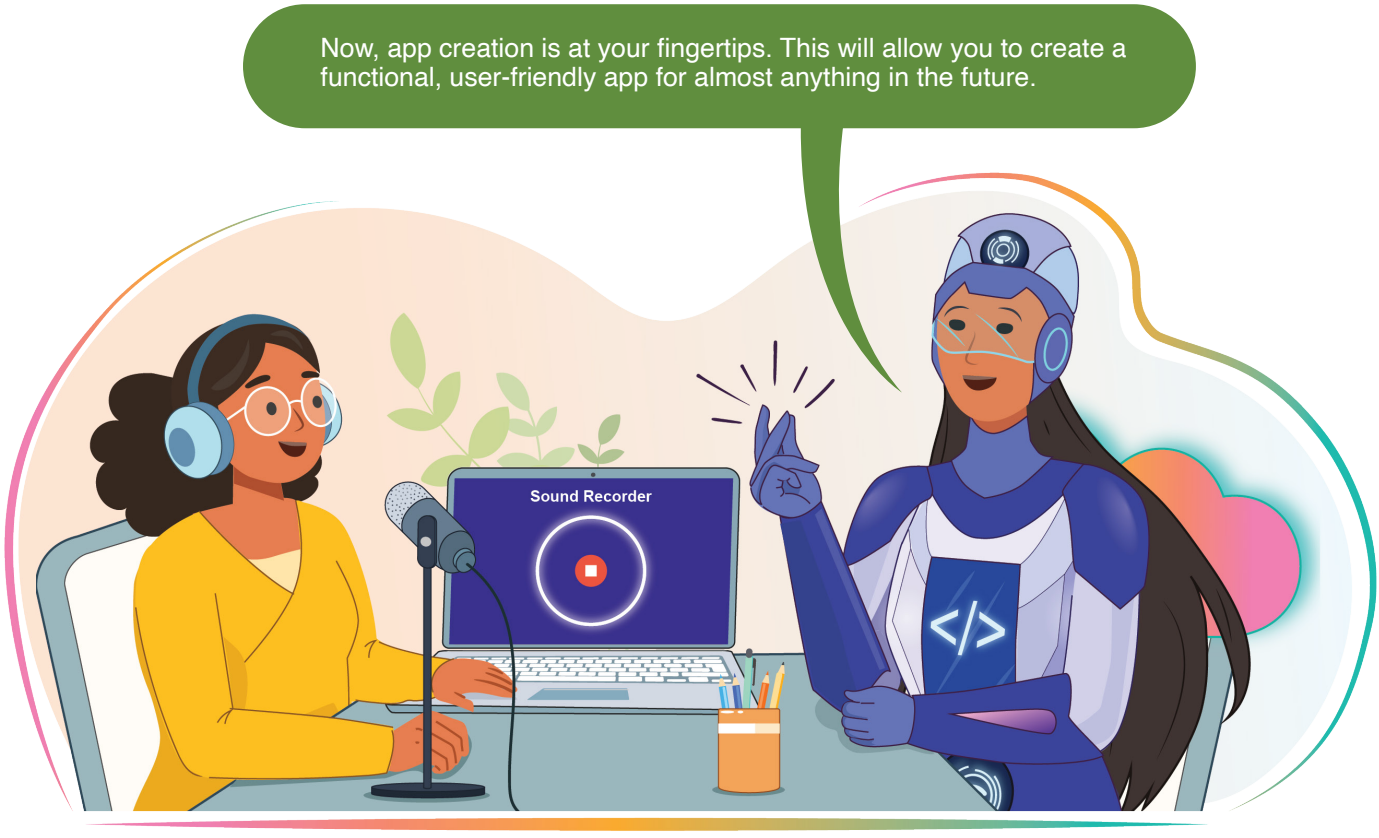
```
42
43    if(recorderState == 1){
44      let ampl = mic.getLevel();
45      let rad = map(ampl,0,0.2,100,500);
46
47      stroke(255);
48      strokeWeight(5);
49      noFill();
50      circle(width/2+32, height/2+32,rad);
```

Now, we can see the changes in the circle's diameter based on the mic input. This simple sound visualizer provides a better appearance to the app as well as indicates that the recording is on.



We have now completed creating a web-based audio recorder by incorporating the concept of UI/UX. This lesson highlights the importance of a smooth and easy experience while using an app. Keeping that in mind, we can try to create our own creative UI/UX for the application.

5. Audio Recorder - Part 2



Exercise

Answer the questions with one sentence

5 What is sound visualization?

_____

_____

6 Which two audio properties can be used for sound visualization?

_____

_____

5. Audio Recorder - Part 2

Tech Flash

- **User Interface (UI)** : The 'User Interface' refers to the graphical layout of an application or device that allows users to interact with it.
- **User Experience (UX)** : 'User Experience' is how a user feels while interacting with various elements of an application.
- **Sound Visualization** : It is the process of converting sound into visually perceptible animations.